

Leakage-Aware Multi-Task Recognition of Produce Freshness and Type for Low-Cost Mobile Deployment

Yashash Mathur

RV Institute of Technology and Management
Bengaluru, India
yashashgdg@gmail.com

Utsav Upadhyay

RV Institute of Technology and Management
Bengaluru, India

Aditya Prakash

RV Institute of Technology and Management
Bengaluru, India

Shresth Modi

RV Institute of Technology and Management
Bengaluru, India

Abstract—Papers on produce freshness classification regularly report accuracy close to 100% on public Kaggle datasets. We took a close look at one of the most popular of these datasets and found that part of that accuracy reflects how the data are split and, more importantly, how the photographs were taken. Its 14,682 images collapse into 1,279 near-duplicate clusters, drawn from roughly 35 capture sessions. A random per-file split puts a copy of 100.0% of the test images into training, and an EfficientNet-B0 then scores 100.0% on freshness. Keeping each cluster on one side of the split gives 98.3%, yet 97.7% of those test images still share a capture session with training images. When whole sessions are held out, freshness accuracy falls to 84.9% and calibration error rises from 1.4% to 12.2%. The failures cluster where fresh and stale photographs of the same produce came from different cameras or messaging apps: on an unseen session of stale capsicum the model is right on only 6.9% of images. On the photo-grouped split, joint training of freshness and produce type performs on par with separate models, and a multi-task MobileNetV3-Large reaches 98.5% freshness accuracy in 16.7 ms on a laptop CPU. An energy-score gate on its produce-type head rejects 97.3% of non-produce images but only 86.4% of unsupported produce. On real phone photos the research model did worse still: it accepted and correctly named only 13.3% of 60 web photos. A retrained model with a two-stage scan, which detects each item and classifies a square crop around it, names the right type in 72.6% of web and user photos. We release the splits, code and an Android app that runs the model offline, and we argue that shelf life and quality grade, for which these datasets hold no ground truth, should not be presented as learned outputs.

Index Terms—food quality, freshness classification, data leakage, multi-task learning, out-of-distribution detection, on-device inference

I. INTRODUCTION

In informal markets, fruit and vegetables are usually sorted by eye. A phone app that flags stale produce from a single photo would be a cheap aid for vendors and shoppers, and public datasets of fresh and rotten (or stale) produce have turned the idea into a popular benchmark. Reported accuracies sit close to 100% [1].

We were suspicious of those numbers for a simple reason. Several of the most widely used Kaggle releases ship *pre-augmented* images: the same photograph appears many times, rotated, flipped, shifted or sprinkled with noise. Split such a dataset at random, file by file, and near-copies of a test image end up in the training set. This is the textbook form of leakage described by Kaufman *et al.* [2], and Kapoor and Narayanan [3] show how widespread it is in applied machine learning.

This paper asks how much of the reported performance survives once that leakage is removed, and then pushes one level further, to the sessions in which the photographs were taken. We make four contributions.

- 1) A leakage audit of the Kaggle “Fresh and Stale Images of Fruits and Vegetables” dataset [4]. Its 14,682 images collapse into 1,279 near-duplicate clusters (1,622 by file name alone), and a per-file split puts a sibling of 100.0% of test images into training. A held-out-session analysis then shows that part of the remaining accuracy depends on how and when the photographs were taken. We release both splits.
- 2) A controlled comparison of multi-task and single-task networks with identical backbones, and of EfficientNet-B0 [5] and MobileNetV3-Large [6], over three seeds.
- 3) An evaluation of post-hoc out-of-distribution (OOD) gates that reject photos of unsupported produce and of things that are not produce at all.
- 4) An Android app that runs the model entirely on the phone and shows quality grade and remaining shelf life, which no available dataset annotates, only as clearly labelled estimates. When it failed on real photos, we added a deployment model and a two-stage scan that detects each item and classifies a crop around it (Section VII).

II. RELATED WORK

Freshness classification. Classifiers trained on public fresh/rotten datasets routinely report accuracies in the high 90s. Transfer-learned AlexNet reaches 99.8% on one such dataset [7], and a multi-task ResNet ensemble predicts freshness and fruit type together at 98.5% and 97.4% [8]. The closest work to ours is Multi-Task NoisyViT [1], which reports 99.85% on the same Fresh/Stale dataset using random 60/20/20 splits; the only deduplication it describes is MD5 hashing, applied when building a merged Freshness44 set. In [7] the images are rotation-augmented *before* a random 80/10/10 split. None of these papers keeps augmented or near-duplicate copies of a photograph on one side of the split, and we could not find a freshness study that measures how much this matters.

Shelf-life estimation. Remaining shelf life cannot be learned without measuring it. Datasets that follow produce over time until it spoils do exist [9], and a recent CNN+DeiT study [10] forecasts spoilage with a symmetric mean absolute percentage error above 40%. Our dataset has no such measurements, so we do not try to learn shelf life.

Multi-task learning. A shared representation can help related tasks [11], [12], but only when the tasks really are related. We test this directly.

OOD detection. Maximum softmax probability (MSP) [13] and the energy score [14] need no retraining, which makes them easy to run on a phone.

Leakage. Kaufman *et al.* [2] formalise leakage in data mining, and Kapoor and Narayanan [3] show how often it inflates published results, with samples duplicated across splits among the causes.

III. DATA AND LEAKAGE AUDIT

A. Datasets

Main set. The Kaggle “Fresh and Stale Images of Fruits and Vegetables” dataset [4] has 14,682 images of six produce types: apple, banana, bitter melon, capsicum, orange and tomato. Each image is labelled fresh or stale by the folder it sits in, and we use both labels as ground truth.

External set. We use the Kaggle “Fruit and Vegetable Image Recognition” dataset [15] (36 classes, no freshness labels) for evaluation only. Its 590 images of apple, banana, orange, tomato, capsicum and bell pepper form a cross-dataset test of produce type, and the 3,206 images of its other classes form a near-OOD set. We drop 29 external images that are near-duplicates of main-set images.

Far-OOD set. We draw 2,000 CIFAR-10 test images [16] at random and upsample them to the model’s input size.

B. Leakage and the grouped split

The file names tell you how each image was made. Prefixes such as `rotated_by_15_`, `saltandpepper_`, `translation_`, `vertical_flip_` and `Copy of_`, and suffixes such as `.jpg_0_4456.jpg`, mark augmented copies of a source photo. We strip these markers, map every file back to its source name, and group files that share a source

TABLE I

GROUPED SPLIT: IMAGES PER PRODUCE TYPE, FRESHNESS AND SPLIT.

Type	Train		Val		Test	
	Fresh	Stale	Fresh	Stale	Fresh	Stale
Apple	1,165	1,741	300	307	228	294
Banana	1,219	1,036	158	223	204	208
Bitter melon	246	294	41	0	40	63
Capsicum	836	860	31	41	123	0
Orange	1,033	1,144	228	192	205	259
Tomato	941	736	40	82	0	164

TABLE II

ACCURACY (%) UNDER A PER-FILE SPLIT VERSUS THE GROUPED SPLIT. Δ = INFLATION CAUSED BY LEAKAGE.

Model	Freshness			Produce type		
	Per-file	Grouped	Δ	Per-file	Grouped	Δ
HSV hist.+LR	94.0	86.3	+7.7	96.6	80.0	+16.6
B0 multi-task	100.0	98.3	+1.7	100.0	99.8	+0.2

within a produce type (1,622 groups). We then compute a 64-bit difference hash of every image and merge groups whose images are within 2 bits of each other, again only within a produce type, because low-texture images of different produce otherwise collide. The result is 1,279 clusters.

Merging is transitive, so a single chance match can join two different photographs. 18 clusters (3,883 images) contain both fresh and stale images, and the largest holds 933 images. Over-merging cannot leak test images into training; it only makes the split coarser. For that reason we call the units clusters rather than photographs.

We assign whole clusters to train, validation and test with stratified group k -fold sampling. Some photo sessions contain hundreds of copies, so a single random draw can starve a class. We therefore try 50 seeds and keep the split whose per-stratum validation and test shares are closest to 15% (seed 49). It has 11,251/1,643/1,788 images and 200 independent test clusters (Table I). For comparison we also draw a per-file split with the same proportions; under it, 100.0% of test images have a sibling in training.

IV. METHOD

A. Model

A timm [17] backbone pretrained on ImageNet [18] (EfficientNet-B0 or MobileNetV3-Large) produces a pooled feature vector, and each task gets its own small head: dropout 0.3, a 256-unit layer with ReLU, dropout 0.2 and a linear output, with two classes for freshness and six for produce type. The multi-task loss is the mean of the two cross-entropy losses. The single-task baselines use the same backbone and head with one task.

B. Heuristic attributes

No dataset we could find contains graded quality or measured shelf life, so we learn neither. The app still shows both as rough guides, derived from the predicted $p = P(\text{fresh})$:

TABLE III

TEST RESULTS ON THE PHOTO-GROUPED SPLIT (CAPTURE SESSIONS ARE STILL SHARED WITH TRAINING), MEAN $\pm$ STD OVER THREE SEEDS (%). CPU LATENCY: BATCH 1, 4 THREADS, MEDIAN OF 200 INTERLEAVED RUNS.

Model	Fresh acc.	Fresh macro-F1	Type acc.	Type macro-F1	Params (M)	CPU (ms)
EfficientNet-B0, multi-task	98.3 $\pm$ 0.1	98.3 $\pm$ 0.1	99.8 $\pm$ 0.2	99.8 $\pm$ 0.2	4.67	22.3
EfficientNet-B0, freshness only	97.6 $\pm$ 0.4	97.6 $\pm$ 0.4	–	–	4.34	24.6
EfficientNet-B0, type only	–	–	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	4.34	22.1
MobileNetV3-L, multi-task	98.5 $\pm$ 0.4	98.5 $\pm$ 0.4	99.7 $\pm$ 0.4	99.8 $\pm$ 0.2	4.86	16.7
HSV histogram + LR	86.3	86.2	80.0	76.0	–	–

TABLE IV

OUT-OF-DISTRIBUTION DETECTION (% , MEAN $\pm$ STD OVER THREE SEEDS). NEAR-OOD: UNSEEN PRODUCE CLASSES FROM AN EXTERNAL DATASET; FAR-OOD: CIFAR-10 TEST IMAGES. HIGHER AUROC AND LOWER FPR@95%TPR ARE BETTER.

Score	Near-OOD (3,206 images)		Far-OOD (2,000 images)	
	AUROC	FPR@95	AUROC	FPR@95
<i>EfficientNet-B0, multi-task</i>				
Neg. entropy, freshness (v1 gate)	91.0 $\pm$ 2.5	37.7 $\pm$ 8.7	95.8 $\pm$ 0.8	25.8 $\pm$ 7.8
MSP, freshness	91.1 $\pm$ 2.5	37.7 $\pm$ 8.7	95.8 $\pm$ 0.8	25.8 $\pm$ 7.8
MSP, produce type	97.8 $\pm$ 0.4	8.8 $\pm$ 1.8	98.9 $\pm$ 0.4	3.7 $\pm$ 2.6
Energy, produce type	98.2 $\pm$ 0.4	7.7 $\pm$ 2.0	99.4 $\pm$ 0.2	2.7 $\pm$ 1.5
<i>MobileNetV3-L, multi-task</i>				
Neg. entropy, freshness (v1 gate)	91.3 $\pm$ 1.6	42.9 $\pm$ 4.6	95.1 $\pm$ 1.5	32.3 $\pm$ 11.8
MSP, freshness	91.3 $\pm$ 1.6	42.9 $\pm$ 4.6	95.1 $\pm$ 1.5	32.3 $\pm$ 11.8
MSP, produce type	96.0 $\pm$ 1.2	19.1 $\pm$ 7.2	98.3 $\pm$ 0.6	6.8 $\pm$ 4.4
Energy, produce type	96.6 $\pm$ 1.6	15.4 $\pm$ 8.4	99.3 $\pm$ 0.5	3.6 $\pm$ 3.1

grade A if $p \geq 0.85$, B if $p \geq 0.5$ and C otherwise, and an indicative shelf life of $d_t p$ days, where d_t is a room-temperature reference for produce type t . Both fields are labelled as estimates, and we make no accuracy claim about them.

C. OOD gate

We compare four post-hoc scores: the negative entropy of the freshness head, which was the gate used by an earlier version of the system; the MSP of the freshness head; the MSP of the produce-type head [13]; and the energy score $\log \sum_k e^{z_k}$ over the produce-type logits, i.e. the negative free energy at temperature 1 [14]. The deployed gate uses the energy score. Its threshold accepts 95% of validation images and is stored with the model.

D. Training and evaluation

Images are resized to 224×224 and normalised with ImageNet statistics. During training we apply random resized crops (scale 0.8–1), horizontal and vertical flips, 90° rotations, colour jitter, Gaussian noise and coarse dropout. We use AdamW [19] with a learning rate of 3×10^{-4} , weight decay 10^{-4} and batch size 64. The first epoch runs at 10% of the base learning rate, followed by cosine decay. Training uses mixed precision and gradient clipping at 1.0, and stops early on validation loss (patience 3) after at most 10 epochs; in practice runs stopped after 5–10 epochs. Every configuration is trained with seeds 0, 1 and 2, and we report the mean and standard deviation. Calibration is measured with the expected calibration error (ECE, 15 bins) [20]. Images from one cluster are not independent, so for a single run we report a 95%

confidence interval from 1,000 bootstrap resamples of test clusters rather than images [21]. As a floor we train a logistic-regression classifier on $8 \times 8 \times 8$ HSV colour histograms.

Training ran on one NVIDIA RTX 4060 Laptop GPU (8 GB). Latency is measured on an Intel Core i7-14700HX with 4 threads and batch size 1, as the median of 200 runs interleaved across all models so that background load affects them equally. We used Python 3.13, PyTorch 2.12, timm 1.0.24, PyTorch Lightning 2.6.1 and Albumentations 2.0.8. Code, splits and per-run logs are available at https://github.com/mathuryashash/FreshTrack_AI.

V. RESULTS

A. Leakage from augmented copies

Fig. 1 and Table II compare the protocols. With a per-file split, every EfficientNet-B0 seed classifies every test image correctly (100.0% on freshness, 100.0% on produce type). Keeping clusters together brings this down only slightly, to 98.3% and 99.8%. The colour-histogram baseline tells a different story. Its freshness accuracy drops from 94.0% to 86.3%, and its produce-type accuracy from 96.6% to 80.0%. On a per-file split, a model that sees nothing but colour looks almost as good as a CNN, which hides how much of the task the CNN is actually solving.

B. Capture sessions

Grouping the copies of each photograph is not enough to make the test set independent of training. Going by the file names, the whole dataset was photographed in roughly 35 capture sessions (a date or a camera series within a class), and 97.7% of the grouped-split test images come from

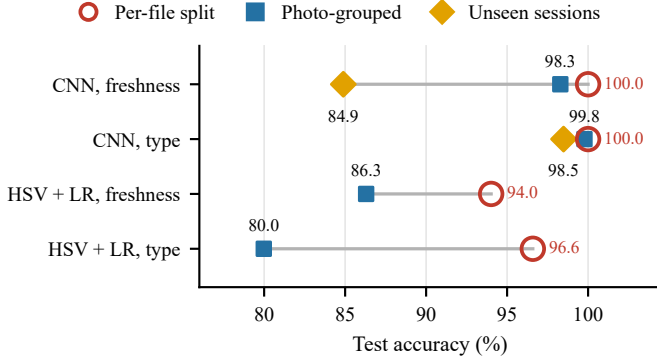


Fig. 1. Test accuracy under three protocols. Open circles: per-file split. Squares: photo-grouped split. Diamonds: unseen capture sessions (EfficientNet-B0 only).

a session that also supplies training images. 4 of the 12 produce/freshness strata come from a single session. For the 8 strata with more than one session we built a second split in which one whole session per stratum is held out for testing (10,310/2,050/2,155 images). We remove the 167 training images that are augmented copies or near-duplicates (within 2 bits) of a test image, so no training image is that close to any test image. Clusters can still cross this split, because chained merging links different photographs from the same session. That dependence is exactly what the protocol is meant to probe.

On unseen sessions, freshness accuracy falls to 84.9% (from 98.3%), while produce-type accuracy holds at 98.5%. Calibration suffers more than accuracy: freshness ECE rises from 1.4% to 12.2%, so the model is often confidently wrong. Fig. 2 shows where the errors come from. Averaged over the 6 strata that also appear in the grouped test, accuracy falls from 98.9% to 87.0%, but the drop is far from uniform. Apple, bitter gourd, orange and stale-tomato sessions transfer almost perfectly. The failures are capsicum (26.6% on an unseen fresh session, 6.9% on an unseen stale one) and fresh tomato (55.2%).

These are precisely the cases in which the held-out session’s capture source appears in training only with the opposite label. Every stale capsicum left in training is a WhatsApp-compressed photograph, and every camera (IMG-series) capsicum in training is fresh. Camera photographs of stale capsicums are duly called fresh. In the same way, all IMG-series tomatoes in training are stale, and the held-out IMG-series fresh tomatoes are often called stale. We read this as a sign that the network has partly learned the acquisition source rather than freshness. Neither a per-file nor a per-photo split can reveal it, since 97.7% of their test images share a session with training images.

C. Multi-task versus single-task, and the choice of backbone

Table III gives the grouped-split results. The multi-task EfficientNet-B0 reaches 98.3% on freshness and 99.8% on produce type; the single-task networks reach 97.6% and 100.0%.

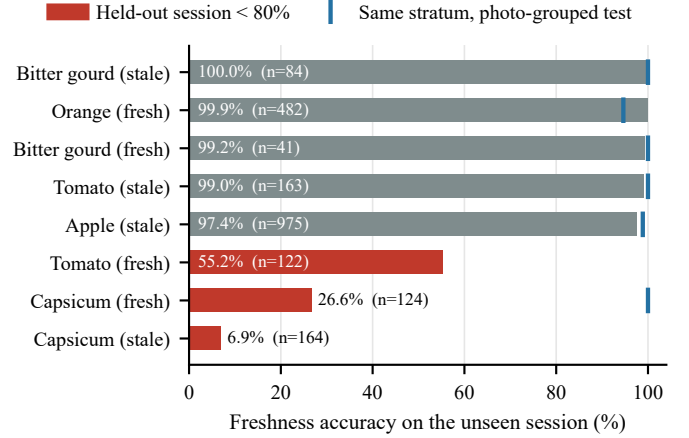


Fig. 2. Freshness accuracy of EfficientNet-B0 (three seeds pooled) on each held-out capture session; n is the number of held-out images. Red bars fall below 80%. Blue ticks mark the same stratum on the photo-grouped test, where that stratum exists.

The gaps are under one percentage point, they point in opposite directions, and three seeds cannot separate them. We see no evidence that joint training helps or hurts, but one shared network does halve the inference cost. MobileNetV3-Large matches EfficientNet-B0 (98.5% and 99.7%) with 4.86 M parameters and runs in 16.7 ms instead of 22.3 ms, so it is the model we deploy. For a single seed-0 run, the 95% group-bootstrap interval of freshness accuracy is [96.4, 99.6]%. It is that wide because only the 200 test clusters, not the 1,788 images, are independent.

D. Cross-dataset produce type

On 590 external images of five of the six produce types (bell pepper counts as capsicum, and there is no bitter gourd), produce-type accuracy is only 62.5% for MobileNetV3 and 63.3% for EfficientNet-B0, against roughly 99% in-domain. The external photos carry no freshness labels. MobileNetV3 calls 83.0% of them fresh and EfficientNet-B0 72.9%, which is plausible for retail photos but is not an accuracy.

E. Out-of-distribution gate

Table IV compares the post-hoc scores, and Fig. 3 shows the energy scores of the deployed model. Scores computed on the six-way produce-type head separate supported from unsupported images much better than scores on the two-way freshness head: near-OOD AUROC is 96.6% for energy on the type head and 91.3% for MSP on the freshness head. With only two classes, entropy is a monotone function of MSP and ranks inputs identically. The earlier version of the system rejected an image when freshness entropy exceeded 1.5 bits or the top probability fell below 0.3. Neither can happen with two classes (entropy is at most 1 bit and the top probability at least 0.5), so that gate never fired.

At a threshold that keeps 95% of validation images, the MobileNetV3 energy gate, averaged over three seeds, accepts 93.5% of in-distribution test images and rejects 97.3% of

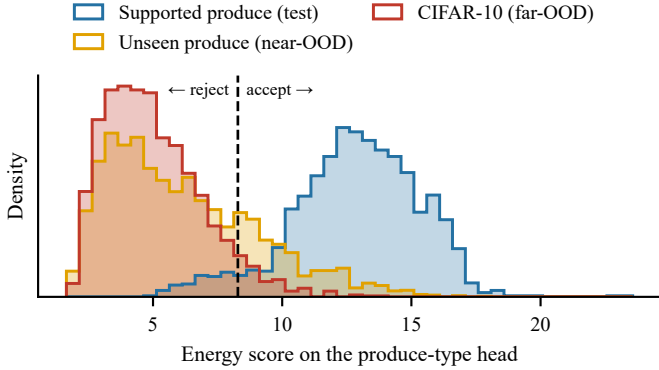


Fig. 3. Energy scores of the deployed model. The dashed line is the gate threshold, set to accept 95% of validation images. Unseen produce overlaps the accepted region far more than CIFAR-10 images do.

CIFAR-10 images, but only 86.4% of unseen produce (77.1–91.4% across seeds; for the deployed seed, 94.5%, 94.2% and 77.1%). Fig. 3 makes the problem visible. Most CIFAR images fall well below the threshold, while unseen fruit and vegetables straddle it. A photo of a vegetable the model has never seen is the hard case.

VI. DEPLOYMENT

This section describes the first release (v2.0.0), which ran the research MobileNetV3-Large; Section VII covers what changed after it met real photos. The app runs the model entirely on the phone, so photos never leave the device and no network connection is needed; the release build does not even request internet access. We export the deployed MobileNetV3-Large to ONNX in 32-bit floating point (19.4 MB) and run it with ONNX Runtime inside a Flutter app. Dynamic 8-bit quantisation would cut the file to 5.1 MB, but it changed many of the model’s predictions, so we did not use it. The model ships with a metadata file holding the label order, preprocessing constants and OOD threshold, which keeps the app and the training code from drifting apart. The app labels the grade and shelf-life fields as estimates and avoids food-safety wording (“looks fresh” rather than “safe to eat”).

We checked the port on 40 held-out test images run through the full on-device pipeline, from JPEG or PNG decoding to the gate decision. Freshness, produce type and the gate decision agreed with PyTorch on every image; the largest logit difference, 0.27, comes from the JPEG decoder rather than the model. The arm64 APK is 52.3 MB (44.9 MB for 32-bit phones), of which the ONNX Runtime library and the model take 19.2 and 19.4 MB. On an Android emulator with two virtual cores, preprocessing and inference take 137–142 ms (median), a full scan in the app about 440 ms, and a cold start 2.2 s. Memory use is 85 MB at idle and 169 MB once the model is loaded, peaking at 327 MB during scans, and each saved scan adds about 46 kB of storage. Emulator timings are only a guide to phone performance, which we did not measure.

TABLE V

CLASSIFIERS AT THE GATES THE APP USED: THE RESEARCH MODEL (SHIPPED IN V2.0.0) AT ITS 95%-VALIDATION GATE, 8.28; V2.0.1 AT 5.0; V2.1 AT 5.34. UPPER ROWS: ACCEPTED WITH THE RIGHT TYPE; LOWER ROWS: ACCEPTED, WHERE LOWER IS BETTER (% , n IN BRACKETS). V2.0.1 AND V2.1 TRAIN ON THE TRAINING SPLITS OF BOTH REAL-WORLD SETS, V2.1 ALSO ON COCO TRAIN2017 CROPS.

Test set	Research	v2.0.1	v2.1
Web photos (60)	13.3	45.0	53.3
Fresh/rotten set, test (613)	34.4	94.8	95.8
Vegetable set, test (600)	1.5	99.5	99.7
Original grouped test (1,788)	94.5	99.2	99.1
External set (590)	27.5	69.5	79.7
COCO val2017 crops (574)	–	28.7	65.0
Unsupported produce, test (5,393)	26.2	30.6	31.7
CIFAR-10 (2,000)	5.9	6.6	4.0

TABLE VI

THE WHOLE APP ON REAL PHOTOS (%). RIGHT TYPE: THE BEST ACCEPTED ITEM IN THE PHOTO HAS THE RIGHT TYPE. FOUND: A DETECTION MATCHES THE BANANA, APPLE OR ORANGE BOX ONE-TO-ONE AT $\text{IoU} \geq 0.5$. AP50 IS CLASS-AGNOSTIC. FALSE ACCEPT: AT LEAST ONE ITEM ACCEPTED. COCO: THE COCO-PRETRAINED SSDLITE, UNCHANGED. n IN BRACKETS.

	Whole photo		Detector + crops	
	v2.0.1	v2.1	COCO	Ours
<i>Right type</i>				
Web and user photos (62)	45.2	53.2	67.7	72.6
COCO single-type scenes (138)	10.9	22.5	41.3	46.4
<i>COCO val2017 fruit boxes (200 scenes)</i>				
Large (244), found	–	–	59.0	55.7
found, right type	–	–	38.9	38.9
Medium (391), found	–	–	6.1	11.3
found, right type	–	–	4.3	8.2
AP50, all produce	–	–	22.1	20.7
<i>False accepts (lower is better)</i>				
Unsupported produce (1,000)	29.6	31.1	31.1	33.9
CIFAR-10 (2,000)	6.6	4.1	4.1	3.5

VII. FROM BENCHMARK TO PHONE PHOTOS

The first release shipped the research model, and it failed on real photos. This section describes the failure and the two changes that followed. The research model and every result above are unchanged.

A. Web photos

The released app (v2.0.0) rejected one of our own photos, bananas held in a hand, as not a fruit. To see whether this was bad luck, we downloaded 61 Creative Commons photos of the six supported types from Openverse, labelled their type and freshness by eye and removed 1 that nearly duplicated a training image. No model is trained on the remaining 60 web photos. At its gate, the research model accepted 25.0% of them and named the type of 43.3%, although it names the type of 99.9% of its own test images. On the test splits of two public sets of real-world photos, a fresh/rotten set of fruit and vegetables [22] and a vegetable set [23], the type was right for only 58.9% and 22.3% of images. Our training photos

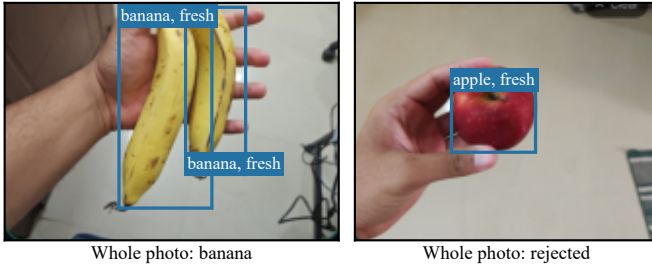


Fig. 4. Our two phone photos through the v2.1 scan. Boxes: detections; labels: type and freshness predicted for each crop, all accepted. Below each photo: the v2.1 classifier on the whole photo.

show single items on plain backgrounds from about 35 capture sessions, and a photo taken in a kitchen looks different.

B. A deployment model (app v2.0.1)

We trained a second model for the app with the same multi-task MobileNetV3-Large. It also learns from the training splits of the two real-world sets; the vegetable set has no freshness labels, so its photos train only the type head. Their photos of other produce form a new set of unsupported produce, split in half, one half for choosing gates and the other for testing. Augmentation is heavier than for the research model, with more aggressive random crops, blur, JPEG recompression and stronger colour jitter. At its own 95%-validation gate the new model still rejected 68.3% of the web photos. Cluttered real photos and photos of unsupported produce get similar energy scores, so no threshold separates them. We lowered the gate to 5.0 after a sweep in which we looked at the web photos, which makes the web results of v2.0.1 optimistic. Table V compares the classifiers at the gates the app used.

C. Why whole photos fail

The next photo to fail showed one apple held in a hand over a floor (Fig. 4, right). The v2.0.1 app rejected it, and the v2.1 classifier described below rejects the whole photo too, but a square crop around the apple passes the gate as an apple. The classifier had learned what a photo looks like when one item fills the frame, and a cluttered photo does not look like that. Crops of real scenes were still hard for it. Of 574 square crops around the annotated bananas, apples and oranges in COCO val2017 [24], v2.0.1 accepted only 28.7% with the right type. The app needed a way to find items, and the classifier needed to see crops of real scenes.

D. Two-stage scan (app v2.1.0)

The app now finds each item first and classifies a square crop around it. The detector is SSDlite from torchvision, with a MobileNetV3-Large backbone [6], [25], pretrained on COCO. We replace its COCO classifier with a single class, produce, initialised from COCO’s background and produce-class weights. With one class the detector only has to find produce; whether an item is a supported type is left to the classifier and its gate. The app keeps up to 8 boxes that score

at least 0.35 after non-maximum suppression, cuts a square of 1.2 times the longer box side around each, and classifies it. The score threshold gives the best F1 at IoU 0.5 on validation (precision 0.90, recall 0.76). If nothing is found, the app classifies the whole photo as before, and users can also draw a box themselves.

Detector data. The detector trains on 23,453 images and is validated on 2,196. They come from four sources: COCO train2017 scenes with produce, using COCO’s boxes for banana, apple, orange, broccoli and carrot and Grounding DINO [26] boxes for produce that COCO has no category for; food-free COCO scenes as negatives; photos from the three Kaggle sets, boxed by Grounding DINO prompted with their known type; and 13,000 synthetic scenes of SAM [27] cutouts pasted onto food-free COCO images, as in cut-and-paste synthesis [28], [29]. A shape filter drops masks that are slivers, fill nearly all of their box or have holes. We checked small samples by eye: most automatic boxes were fully right and the rest partly right, and the filter removed most faulty cutouts and none of the good ones. None of the detector’s data comes from COCO val2017, the web photos or the classifier’s test splits. AP50 on a fixed validation subset peaked at 84.7 after epoch 4 of 10.

Classifier v2.1. We fine-tuned the v2.0.1 classifier for 8 epochs on its own data plus 12,600 crops cut the way the app cuts them: 6,000 around COCO train2017 boxes, with type labels only, and 6,600 from the synthetic scenes, with the type and, where the source photo has one, the freshness label.

Gates. Both v2.1 gates were set on the validation half of the unsupported produce, by matching false accepts. The whole-photo gate, 5.34, accepts the same share of these photos as v2.0.1 did at 5.0. Crops score higher than the photo they come from: at the whole-photo gate, the detector’s crops would accept 31.0% of 1,000 validation photos, against 28.3% for whole photos. Crops therefore get their own gate, set so that the two rates match. A photo counts as accepted when any of its items passes.

E. Results

Fine-tuning on crops raised the share of COCO crops accepted with the right type from 28.7% to 65.0% (Table V). It also helped on whole web photos and on the external set. The original test split stayed at 99.1%, and fewer CIFAR-10 images passed the gate.

Table VI compares whole apps. As a baseline for our detector we ran the COCO-pretrained SSDlite unchanged, counting its banana, apple and orange classes as produce, with a score threshold of 0.30 fixed in advance and a crop gate calibrated in the same way. On the 62 web and user photos, the best accepted item has the right type in 45.2% of photos with the v2.0.1 app, 53.2% with the v2.1 classifier on whole photos and 72.6% with the two-stage scan. On the 138 COCO scenes that contain a single supported type, the share rises from 10.9% to 46.4%. When the type is right, the predicted freshness agrees with our visual label on 93.0% of the 43 web photos concerned.

Items that fill little of the frame are the weak spot. SSDlite sees the photo at 320×320 pixels, and our detector finds 55.7% of the large supported fruit in COCO scenes but only 11.3% of the medium-sized ones. Its class-agnostic AP50 over all annotated produce is 20.7. That is a lower bound, because COCO does not annotate tomatoes, peppers or lemons, and finding one counts as a false positive. The classifier loses items too. Given the annotated boxes as crops, as a perfect detector would supply them, it accepts the right type for 63.9% of large and 60.6% of medium fruit, so for medium fruit most of the loss is in detection.

False accepts on unsupported produce went up. With both gates matched on validation, the two-stage app accepts at least one item in 33.9% of 1,000 test photos of unsupported produce, against 29.6% for v2.0.1. On CIFAR-10, false accepts fall from 6.6% to 3.5%.

On an Android emulator (profile build), the app’s detector agreed with the Python reference on 5 test photos: the same number of items in each photo, 14 in all, with boxes within 0.8 pixels and the same type and gate decision for every item. On its own, the on-device classifier matched PyTorch on type and gate for all 40 parity photos and on freshness for 100.0 of them. Median times per photo were 224 ms to decode, 46 ms to detect and 92 ms to classify all crops. The detector’s ONNX file is 8.9 MB.

VIII. LIMITATIONS

Both units of analysis are approximate. Capture sessions are inferred from file names, and hash-based clustering merges some distinct photographs. The held-out-session split covers only the 8 strata that have more than one session; for the rest we have no session-independent estimate, and the grouped-split numbers should be read as upper bounds. Where the fresh and stale images of a produce type came from different devices or apps, freshness and acquisition source are confounded. We can expose that confound, but removing it needs new data. The freshness labels are the dataset authors’ binary folder labels, with no inter-rater agreement and no intermediate ripeness stages. Most images show a single item on a plain background. Section VII measures what this costs on web and COCO photos, but we did not collect field images in markets. CIFAR-10 images are small and upsampled, which makes them an easy far-OOD test. Our latency figures, including those of the two-stage scan, come from a laptop CPU and an Android emulator rather than a range of real phones. Finally, the quality grade and shelf life shown to users are unvalidated heuristics.

The phone-photo tests are small or only partly labelled. The web set has 60 photos plus two of our own, and we chose the v2.0.1 gate while looking at them, so its web results are optimistic. The v2.1 gates were set on validation data, but the false-accept rate they match is the one v2.0.1 reached at that gate. The detector’s COCO AP is a lower bound, because produce that COCO does not annotate, such as tomatoes and peppers, counts against it. At 320 pixels SSDlite misses small items in crowded scenes and finds only 11.3% of the medium COCO fruit. False accepts on unsupported produce

were matched at 28.3% on validation but reach 33.9% on the test half.

IX. CONCLUSION

Near-perfect accuracy on public fresh/stale benchmarks says more about the evaluation protocol than about the models. Removing augmented copies changes little for a CNN and a great deal for a weak baseline. Holding out whole capture sessions exposes a larger drop, concentrated where the photographs of fresh and stale produce came from different sources, and neither per-image nor per-photo splits can show it. On the photo-grouped split, one multi-task MobileNetV3-Large recognises produce type and freshness as well as separate EfficientNet-B0 models at lower cost, runs offline on a phone, and rejects most images that are not produce. Real phone photos repeated the lesson of the capture sessions: the research model rejected or misnamed most of them. Retraining on real-world photos and classifying a crop around each detected item closed much of that gap, but the app still accepts unsupported produce in 33.9% of test photos. The next step is data rather than architecture. Photographs from many sessions, places and devices, taken until the produce spoils, are needed before shelf life can be predicted from RGB images at all.

ACKNOWLEDGMENT

The authors thank the creators of the public datasets used in this study.

REFERENCES

- [1] S. Esfandiari Fard, T. Ghosh, and E. Sazonov, “Multi-task NoisyViT for enhanced fruit and vegetable freshness detection and type classification,” *Sensors*, vol. 25, no. 19, p. 5955, 2025.
- [2] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [3] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, p. 100804, 2023.
- [4] R. R. Potdar, “Fresh and stale images of fruits and vegetables,” <https://www.kaggle.com/datasets/raghavpotdar/fresh-and-stale-images-of-fruits-and-vegetables>, 2021, kaggle dataset, CC0 license. Accessed: 2026-09-25.
- [5] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 6105–6114. [Online]. Available: <https://proceedings.mlr.press/v97/tan19a.html>
- [6] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le, “Searching for MobileNetV3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2019, pp. 1314–1324.
- [7] U. Amin, M. I. Shahzad, A. Shahzad, M. Shahzad, U. Khan, and Z. Mahmood, “Automatic fruits freshness classification using CNN and transfer learning,” *Applied Sciences*, vol. 13, no. 14, p. 8087, 2023.
- [8] J. Kang and J. Gwak, “Ensemble of multi-task deep convolutional neural networks using transfer learning for fruit freshness classification,” *Multimedia Tools and Applications*, vol. 81, no. 16, pp. 22355–22377, 2022.
- [9] O. Abayomi-Alli and R. Damaševičius, “Fruit quality datasets (FruQ-DB),” <https://zenodo.org/records/7224690>, 2022, zenodo, version v1. Accessed: 2026-09-25.
- [10] M. Kanulla, R. Dadigi, S. Thota, and V. Yelleti, “Transformer based multi-task fusion network for food spoilage detection and shelf life forecasting,” arXiv preprint arXiv:2601.13665, 2026. [Online]. Available: <https://arxiv.org/abs/2601.13665>

- [11] R. Caruana, "Multitask learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997.
- [12] S. Ruder, "An overview of multi-task learning in deep neural networks," arXiv preprint arXiv:1706.05098, 2017. [Online]. Available: <https://arxiv.org/abs/1706.05098>
- [13] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1610.02136>
- [14] W. Liu, X. Wang, J. Owens, and Y. Li, "Energy-based out-of-distribution detection," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 21 464–21 475. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/f5496252609c43eb8a3d147ab9b9c006-Abstract.html>
- [15] K. Seth, "Fruits and vegetables image recognition dataset," <https://www.kaggle.com/datasets/kritikseth/fruit-and-vegetable-image-recognition>, 2022, kaggle dataset, CC0 license. Accessed: 2026-09-25.
- [16] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
- [17] R. Wightman, "PyTorch image models," <https://github.com/huggingface/pytorch-image-models>, 2019, gitHub repository. Accessed: 2026-09-25.
- [18] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2009, pp. 248–255.
- [19] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>
- [20] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1321–1330. [Online]. Available: <https://proceedings.mlr.press/v70/guo17a.html>
- [21] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.
- [22] M. Mukhiddinov, "Fruits and vegetables dataset," <https://www.kaggle.com/datasets/muhridinmuxiddinov/fruits-and-vegetables-dataset>, 2022, kaggle dataset, CC0 license. Accessed: 2026-09-26.
- [23] M. I. Ahmed, "Vegetable image dataset," <https://www.kaggle.com/datasets/misrakahmed/vegetable-image-dataset>, 2021, kaggle dataset, CC BY-SA 4.0 license. Accessed: 2026-09-26.
- [24] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, ser. Lecture Notes in Computer Science, vol. 8693. Springer, 2014, pp. 740–755.
- [25] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," in *Computer Vision – ECCV 2016*, ser. Lecture Notes in Computer Science, vol. 9905. Springer, 2016, pp. 21–37.
- [26] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science, vol. 15105. Springer, 2024, pp. 38–55.
- [27] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 3992–4003.
- [28] D. Dwibedi, I. Misra, and M. Hebert, "Cut, paste and learn: Surprisingly easy synthesis for instance detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2017, pp. 1310–1319.
- [29] G. Ghiasi, Y. Cui, A. Srinivas, R. Qian, T.-Y. Lin, E. D. Cubuk, Q. V. Le, and B. Zoph, "Simple copy-paste is a strong data augmentation method for instance segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2021, pp. 2917–2927.